

Mission Statement zur Bachelorarbeit

“Deep Space Debugger - Prototypische Entwicklung eines 2D Serious Games zur spielerischen Vermittlung des objektorientierten Paradigmas”

1. Einleitung und Problemstellung

Das objektorientierte Paradigma (OOP) stellt einen zentralen Bestandteil der Softwareentwicklung dar und ist ein fester Bestandteil der Informatikstudiengänge an der Hochschule für Technik, Wirtschaft und Kultur Leipzig. Dennoch haben Studierende häufig Schwierigkeiten, abstrakte Konzepte wie Klassen, Vererbung und Polymorphie nachhaltig zu verstehen und praktisch anzuwenden.

Traditionelle Lehrmethoden, insbesondere der Frontalunterricht, fokussieren häufig die theoretische Vermittlung und textbasierte Programmierung, wodurch der Bezug zur praktischen Anwendung für Lernende deutlich erschwert wird.

2. Zielsetzung

Ziel dieser Bachelorarbeit ist die Konzeption und Entwicklung eines 2D Serious Games, das grundlegende Konzepte des objektorientierten Paradigmas spielerisch vermittelt.

Das Serious Game „Deep Space Debugger“ soll es ermöglichen, OOP-Prinzipien durch interaktive Problemlösung in einer spielerischen Umgebung erfahrbar zu machen, ohne dass klassische Programmierkenntnisse erforderlich sind.

3. Forschungsfrage

Inwiefern können grundlegende Konzepte des objektorientierten Paradigmas durch ein 2D Serious Game effektiv vermittelt und verständlich gemacht werden?

4. Konzeption des Serious Games

Grundidee	Spieler:in übernimmt die Rolle eines Systemtechnikers in einer beschädigten Raumstation. Die Station besteht aus modularen Systemen (z.B. Türen, Terminals, Lebenserhaltungssysteme), deren Logik fehlerhaft ist.
Genre	○ Serious Game (didaktische Fokussierung) ○ 2D-Puzzle-Game ○ Systemsimulation
Lerninhalte	Das Serious Game vermittelt folgende OOP-Konzepte: ○ Klassen und Objekte ○ Vererbung ○ Polymorphie ○ Objektbeziehungen und Komposition
Mechaniken	1. Probleme erkennen: Spieler:in betritt Räume der Raumstation und erkennt fehlerhaftes Verhalten der Systeme. (z. B. Türen, Strom oder KI funktionieren nicht korrekt). 2. Systeme verbinden oder trennen: Spieler:in stellt Beziehungen zwischen Systemen her oder löst falsche Verbindungen, z. B. zwischen Energieversorgung und Türen. 3. System-Typ ändern: Objekte können in verschiedene Varianten umgewandelt werden (z. B. normale Tür, Sicherheitstür, Notfalltür), wodurch sich ihr Verhalten ändert. 4. Aktionen auslösen: Spieler:in aktiviert Systeme. Je nach Objekttyp führt dieselbe Aktion zu unterschiedlichen Ergebnissen (z. B. Tür öffnet sich, Alarm wird ausgelöst, KI reagiert). 5. Debug-Menü (Interface): Ein vereinfachtes Interface ermöglicht das Anpassen von Systemzuständen und das Reparieren von Logikfehlern ohne klassischen Code.

Technische Umsetzung

Konzeptuelles Modell:

Die Konzeption orientiert sich am MDA-Framework, welches zur strukturierten Entwicklung und Analyse von Spielen verwendet wird.

Technologien:

- Godot Engine
- GDScript
- UI-System für Debugging-Interface

Architektur:

Das Spiel basiert auf einem modularen Objektmodell, das die Prinzipien der objektorientierten Programmierung direkt im Game Design widerspiegelt.